

SQL INTERVIEW HANDBOOK

50 MUST-KNOW CONCEPTS

1 SQL FUNDAMENTALS

★ What is SQL?

SQL (Structured Query Language) is a standard language used to create, manage and manipulate relational databases.

★ What is Database?

A Database is a collection of related data.

★ What is DBMS?

DBMS (Database Management System) is software that helps to store, retrieve, update and manage databases.

DB vs DBMS

DATABASE	DBMS
- Collection of data. - Stores actual data. - Example: MySQL Database, StudentDB	- Software to manage databases. - Helps in storing, retrieving, updating data. - Example: MySQL, Oracle, SQL Server

What is Table?

A Table is a collection of related data in the form of rows and columns. →

Example: Students

id	name	age	city
1	Aman	21	Delhi
2	Priya	22	Pune
3	Rohan	23	Mumbai

Columns

Rows

★ Key Terms

- Row (Record): Horizontal entry in a table.
- Column (Field): Vertical entry in a table.
- Primary Key: Uniquely identifies each row in a table.
- Duplicate Values: Allowed in normal columns, not in Primary Key.

Basic SQL Query Structure

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

This is the foundation of almost every SQL query!



★ Practice is the key to master SQL ♥

2 KEYS & CONSTRAINTS

Constraints help to enforce rules on data in a table.

TYPES OF KEYS & CONSTRAINTS

CONSTRAINT	DESCRIPTION	EXAMPLE
★ PRIMARY KEY (PK) 	Uniquely identifies each row in a table. Cannot be NULL.	id INT PRIMARY KEY
★ FOREIGN KEY (FK) 	References the Primary Key of another table. Maintains referential integrity.	user_id INT, FOREIGN KEY (user_id) REFERENCES Users(id)
★ UNIQUE 	Ensures all values in a column (or set of columns) are different.	email VARCHAR(100) UNIQUE
★ NOT NULL 	Ensures a column cannot have NULL value.	name VARCHAR(50) NOT NULL
★ CHECK 	Ensures that the values in a column satisfy a specific condition.	age INT CHECK (age >= 18)
★ DEFAULT 	Sets a default value for a column when no value is provided.	status VARCHAR(10) DEFAULT 'ACTIVE'

EXAMPLE :

```
CREATE TABLE Employees (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE,
  age INT CHECK (age >= 18),
  department_id INT,
  FOREIGN KEY (department_id) REFERENCES Departments(id),
  status VARCHAR(10) DEFAULT 'ACTIVE'
);
```

Why Important?

- Maintain data accuracy
- Prevent invalid data
- Ensure relationships between tables
- Improve overall data reliability



NOTE : A table can have only ONE PRIMARY KEY, but can have multiple UNIQUE, CHECK, DEFAULT and FOREIGN KEYS.

Strong Constraints = Strong Database ❤

3 SELECT ESSENTIALS

SELECT is used to retrieve data from a table.

★ 1. BASIC SELECT

Select all columns

```
SELECT *
FROM employees;
```

★ 4. DISTINCT

Returns only unique values

```
SELECT DISTINCT department
FROM employees;
```

★ 2. SELECT SPECIFIC COLUMNS

Select only required columns

```
SELECT id, name, email
FROM employees;
```

★ 5. ORDER BY

Sort the result set

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

ASC =
Ascending
(default)

DESC =
Descending

★ 3. ALIASES (AS)

Rename column or table using alias

```
SELECT name AS employee_name,
       salary AS emp_salary
FROM employees AS e;
```

★ 6. LIMIT / FETCH

Limit number of rows returned

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5; -- Top 5 highest salary
```

EXAMPLE TABLE : employees

id	name	department	salary
1	Aman	IT	90000
2	Priya	HR	70000
3	Rohan	IT	85000
4	Sneha	Finance	60000
5	Karan	IT	95000

SAMPLE QUERIES & OUTPUT

- `SELECT name FROM employees;`
→ Aman, Priya, Rohan, Sneha, Karan
- `SELECT DISTINCT department FROM employees;`
→ IT, HR, Finance
- `SELECT name, salary FROM employees ORDER BY salary DESC;`
→ Karan, Aman, Rohan, Priya, Sneha
- `SELECT name FROM employees LIMIT 3;`
→ Aman, Priya, Rohan

Quick Tips

- Use explicit columns instead of * in production queries.
- Always use `ORDER BY` with `LIMIT` for predictable results.
- Aliases make queries easy to read.

REMEMBER

SELECT lets you see the data.

Good SELECT = Good Query



4 Filtering & Operators

Used to filter, compare and handle conditions in SQL.

1. WHERE Clause

Filters records based on condition.

```
SELECT *
FROM employees
WHERE department = 'IT';
```

2. Comparison Operators

Operator	Meaning	Operator	Meaning
=	Equal to	>=	Greater or equal
<>	Not equal to	<=	Less or equal
>	Greater than	BETWEEN	In a range
<	Less than	IN	In a list

3. Logical Operators

Operator	Description
AND	Both conditions true
OR	Any one condition true
NOT	Reverses the condition

Example :

```
SELECT name, salary
FROM employees
WHERE department = 'IT' AND salary > 70000;
```

4. IN Operator

Matches any value in a list.

```
SELECT name FROM employees
WHERE department IN ('IT', 'HR', 'Finance');
```

5. BETWEEN Operator

Checks value in a given range.

```
SELECT * FROM employees
WHERE salary BETWEEN 60000 AND 90000;
```

6. LIKE Operator

Used for pattern matching with wildcards.

Wildcard	Meaning
%	Any sequence of characters
_	Single character

Example :

```
SELECT name FROM employees
WHERE name LIKE 'A%'; -- Starts with A

SELECT name FROM employees
WHERE name LIKE '%an%'; -- Contains 'an'
```

7. IS NULL / IS NOT NULL

Checks for NULL values.

```
SELECT name FROM employees
WHERE manager_id IS NULL;
```

8. CASE Statement

Conditional logic in queries.

```
SELECT name, salary,
CASE
  WHEN salary >= 90000 THEN 'High'
  WHEN salary >= 70000 THEN 'Medium'
  ELSE 'Low'
END AS salary_category
FROM employees;
```

Output :

name	salary	salary_category
Aman	95000	High
Priya	75000	Medium
Rohan	60000	Low



Remember

- ✓ WHERE → Filters rows
- ✓ HAVING → Filters groups
- ✓ ORDER BY → Sorts result
- ✓ LIMIT → Restricts rows
- ✓ CASE → For if-else logic

Master these operators → Write Better & Smarter SQL Queries! ❤️

5

SQL FUNCTIONS

Functions help to perform calculations, manipulate data and return useful results.

★ 1. AGGREGATE FUNCTIONS

Perform calculation on multiple rows and return a **single value**.

FUNCTION	DESCRIPTION
COUNT()	Returns number of rows
COUNT(column)	Counts non-NULL values
SUM()	Returns total sum
AVG()	Returns average
MIN()	Returns minimum value
MAX()	Returns maximum value

Example :

```
SELECT department,
       COUNT(*) AS total_emp,
       AVG(salary) AS avg_salary,
       MAX(salary) AS max_salary,
       MIN(salary) AS min_salary
FROM employees
GROUP BY department;
```

★ 3. DATE & TIME FUNCTIONS

Work with date and time values.

FUNCTION	DESCRIPTION
NOW() / CURRENT_TIMESTAMP	Current date and time
DATE()	Extracts date part
TIME()	Extracts time part
YEAR(date)	Extracts year
MONTH(date)	Extracts month (1-12)
DAY(date)	Extracts day (1-31)
DATEDIFF(d1, d2)	Difference in days (d1-d2)

Example :

```
SELECT name, hire_date,
       YEAR(hire_date) AS hire_year,
       MONTH(hire_date) AS hire_month,
       DATEDIFF(CURDATE(), hire_date) AS days_worked
FROM employees;
```

TIPS

- Aggregate functions ignore **NULL** values (except COUNT(*))
- String functions are **case sensitive**.
- Always check **data type** before using functions.

FUNCTIONS

Make your queries powerful and smart!



★ 2. STRING FUNCTIONS

Used to manipulate text data.

FUNCTION	DESCRIPTION
UPPER(str)	Converts to upper case
LOWER(str)	Converts to lower case
LENGTH(str)	Returns length of string
SUBSTRING(str, n, m)	Returns substring
TRIM(str)	Removes leading / trailing spaces
CONCAT(str1, str2)	Joins two strings

Example :

```
SELECT UPPER(name) AS name_upper,
       LENGTH(email) AS email_len,
       SUBSTRING(name, 1, 3) AS short_name,
       CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;
```



SUBSTRING(str, start, length)

start = position to start from (1 based)

length = number of characters

★ 4. NUMERIC FUNCTIONS

Used for numeric calculations.

FUNCTION	DESCRIPTION
ROUND(num, n)	Rounds to n decimal places
CEIL(num)	Rounds up to nearest integer
FLOOR(num)	Rounds down to nearest integer
ABS(num)	Absolute value

Example :

```
SELECT salary,
       ROUND(salary, 2) AS salary_2dp,
       CEIL(salary) AS salary_ceil,
       FLOOR(salary) AS salary_floor,
       ABS(-salary) AS abs_salary
FROM employees;
```

QUICK EXAMPLE (ALL IN ONE)

```
SELECT UPPER(name) AS name_upper,
       COUNT(*) AS total_emp,
       AVG(salary) AS avg_salary,
       YEAR(hire_date) AS hire_year
FROM employees
WHERE salary > 50000
GROUP BY YEAR(hire_date);
```

6 GROUP BY & HAVING

Used to group rows that have the same values and filter groups using conditions.

★ 1. GROUP BY

Groups rows based on one or more columns so aggregate functions can be applied.

Example :

```
SELECT department,
COUNT(*) AS total_emp,
AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

Output :

department	total_emp	avg_salary
IT	4	69999.75
HR	3	56666.67
Finance	3	63333.33

★ 2. HAVING CLAUSE

Filters groups after aggregation.

(WHERE filters rows before grouping)

Example :

```
SELECT department,
COUNT(*) AS total_emp,
AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING COUNT(*) > 3;
```

Output :

department	total_emp	avg_salary
IT	4	69999.75

★ 3. GROUP BY + WHERE vs HAVING

WHERE	HAVING
• Filters individual rows	• Filters groups
• Used BEFORE GROUP BY	• Used AFTER GROUP BY
• Cannot use aggregate functions	• Can use aggregate functions
• Improves performance	• Works on grouped result

★ 4. Rules to Remember

- Every column in SELECT (not used in aggregate) must be in GROUP BY.
- HAVING can use aggregate functions, WHERE cannot.
- Order of execution:

FROM → WHERE → GROUP BY →
HAVING → SELECT → ORDER BY

★ 5. Common Interview Questions & Patterns

① Departments with more than N employees

```
SELECT department, COUNT(*) AS cnt
FROM employees
GROUP BY department
HAVING COUNT(*) > 3;
```

② Departments with avg salary > 60000

```
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

③ Employees count in each department

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

④ Multiple conditions in HAVING

```
SELECT department,
COUNT(*) AS cnt, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING COUNT(*) > 2 AND AVG(salary) > 50000;
```

⑤ Top departments by total salary

```
SELECT department, SUM(salary) AS total_sal
FROM employees
GROUP BY department
ORDER BY total_sal DESC
LIMIT 3;
```



TIP

- Use WHERE to reduce data early.
- Use GROUP BY to summarize data.
- Use HAVING to filter summary.

7

JOINS - PART 1

Joins combine rows from two or more tables based on a related column.

EMPLOYEES (e)

id	name	dept_id	salary
1	Aman	10	70000
2	Priya	20	60000
3	Rohan	10	80000
4	Sneha	30	65000
5	Karan	NULL	50000

DEPARTMENTS (d)

id	dept_name
10	IT
20	HR
30	Finance
40	Marketing

KEY

- e.dept_id → Foreign Key
- d.id → Primary Key

★ 1. INNER JOIN

Returns only matching rows from both tables.

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.id;
```

Output :

name	dept_name
Aman	IT
Priya	HR
Rohan	IT
Sneha	Finance

Rows with no match in either table are excluded.

★ 2. LEFT JOIN (or LEFT OUTER JOIN)

Returns all rows from LEFT table and matching rows from RIGHT table.

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.id;
```

Output :

name	dept_name
Aman	IT
Priya	HR
Rohan	IT
Sneha	Finance
Karan	NULL

All rows from LEFT table are returned.



★ 3. RIGHT JOIN (or RIGHT OUTER JOIN)

Returns all rows from RIGHT table and matching rows from LEFT table.

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.id;
```

Output :

name	dept_name
Aman	IT
Priya	HR
Rohan	IT
Sneha	Finance
NULL	Marketing

All rows from RIGHT table are returned.



★ 4. FULL JOIN (or FULL OUTER JOIN)

Returns all rows when there is a match in either table.

```
SELECT e.name, d.dept_name
FROM employees e
FULL JOIN departments d
ON e.dept_id = d.id;
```

Output :

name	dept_name
Aman	IT
Priya	HR
Rohan	IT
Sneha	Finance
Karan	NULL
NULL	Marketing

All rows from BOTH tables are returned.



Note :

- INNER JOIN is most common.
- LEFT JOIN is widely used in reporting.

- Choose join based on which table's data should never be lost.

Understand the Joins Today, Write Powerful Queries Tomorrow!



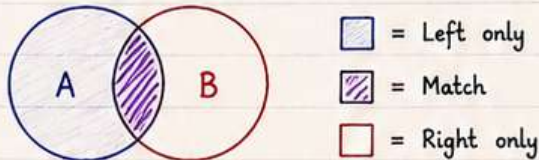
8

JOINS - PART 2

More types of JOINS that are important in interviews.

★ 1. FULL JOIN (or FULL OUTER JOIN)

Returns all rows when there is a match in either LEFT or RIGHT table.



Example :

```
SELECT e.name, d.dept_name
FROM employees e
FULL JOIN departments d
ON e.dept_id = d.id;
```

Output :

name	dept_name
Aman	IT
Priya	HR
Rohan	IT
Sneha	Finance
Karan	NULL
NULL	Marketing

Includes all rows from both tables (matched + unmatched)



★ 2. CROSS JOIN

Returns the Cartesian product of both tables. Every row from table A is combined with every row from table B.



Example :

```
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN departments d;
```

Output (5 employees × 4 departments = 20 rows)

name	dept_name
Aman	IT
Aman	HR
Aman	Finance
...	...
Karan	Marketing

Use CROSS JOIN carefully. It can create a very large result set!



★ 3. SELF JOIN

A table is joined with itself. Useful for hierarchical data (like employee-manager relationship).

Example Table : employees

id	name	manager_id
1	Aman	NULL
2	Priya	1
3	Rohan	1
4	Sneha	2
5	Karan	2



Example (Find employee with their manager):

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.id;
```

Output :

employee	manager
Aman	NULL
Priya	Aman
Rohan	Aman
Sneha	Priya
Karan	Priya

Self JOIN is very common in hierarchy related questions.



★ 4. NATURAL JOIN

Automatically joins tables based on columns with the same name in both tables.

Example :

```
SELECT *
FROM employees
NATURAL JOIN departments;
```

Use when both tables have same column names.



QUICK COMPARISON

JOIN TYPE	WHAT IT RETURNS
INNER JOIN	Only matching rows
LEFT JOIN	All rows from LEFT + matches
RIGHT JOIN	All rows from RIGHT + matches
FULL JOIN	All rows from BOTH tables
CROSS JOIN	Cartesian product (m x n rows)
SELF JOIN	A table joined with itself
NATURAL JOIN	Join on same column names automatically



Understanding JOINS is a must for SQL interviews. → Practice with more examples!



9

SUBQUERIES

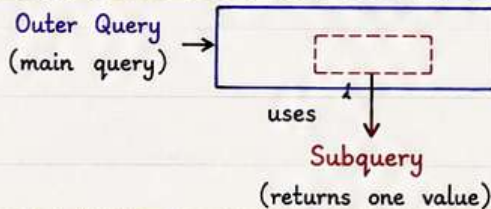
A subquery (inner query) is a query inside another query. Subqueries help break complex problems into smaller parts.

★ 1. SCALAR SUBQUERY

Returns exactly one value (single row, single column).
Used with =, >, <, >=, <=, <> operators.

Example :

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary)
                FROM employees);
```



★ 2. MULTI-ROW SUBQUERY

Returns multiple rows (single column).
Used with IN, NOT IN, ANY, ALL operators.

Example :

```
SELECT name
FROM employees
WHERE dept_id IN (SELECT dept_id
                 FROM departments
                 WHERE location = 'Pune');
```



Operators

IN → matches any value in the list
NOT IN → does not match any value
ANY → compare with any value
ALL → compare with all values

★ 3. CORRELATED SUBQUERY

Refers to columns from the outer query.
Executed once for each row of the outer query.

Example :

```
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary)
                 FROM employees
                 WHERE dept_id = e.dept_id);
```

How it works ?

- ① Outer query picks a row (e.dept_id).
- ② Subquery runs using that value.
- ③ Compares and returns result.

Runs many times (once per outer row).



★ 4. EXISTS & NOT EXISTS

Checks existence of rows.
Returns TRUE if subquery returns any row.

Example :

```
SELECT name
FROM employees e
WHERE EXISTS (SELECT 1
             FROM departments d
             WHERE d.id = e.dept_id);
```

- EXISTS → returns TRUE if at least one row exists.
- NOT EXISTS → returns TRUE if no rows exist.
- SELECT 1 → common practice (we just need to know if row exists, not the data).

★ 5. WHERE vs HAVING with SUBQUERY

Clause	Use With Subquery	Example
WHERE	Filters individual rows (before aggregation)	SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
HAVING	Filters groups (after aggregation)	SELECT dept_id, AVG(salary) FROM employees GROUP BY dept_id HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);

TIPS

- Prefer JOIN over subquery for better performance (when possible).
- Use EXISTS instead of IN for large datasets.
- Always test subqueries separately first.



Golden Rule

If a problem can be solved using a subquery, you can usually solve it using JOIN too!



Break it down. Write it clean. Think in SQL!



10

SUBQUERIES – ADVANCED

Some advanced use cases of subqueries that simplify complex logic and improve query readability.

★ 1. SUBQUERY IN SELECT

Use a subquery to compute a value and show it as a column.

Example :

```
SELECT e.name, e.salary,
       (SELECT AVG(salary) FROM employees) AS avg_salary
FROM employees e;
```

Output :

name	salary	avg_salary
Aman	70000	63750
Priya	60000	63750
Rohan	80000	63750
...		...

Subquery runs once and same value is shown for all rows. 😊

★ 2. SUBQUERY IN FROM (DERIVED TABLE)

Use a subquery in FROM clause as a virtual table.

Example :

```
SELECT d.dept_name, d.avg_sal
FROM (SELECT dept_id, AVG(salary) AS avg_sal
      FROM employees
      GROUP BY dept_id) d;
```

Output :

dept_name	avg_sal
IT	75000
HR	60000
Finance	65000
Marketing	50000

The subquery creates a derived table 'd' and outer query reads from it. 😊

★ 3. SUBQUERY WITH ANY / ALL

ANY / ALL are used with comparison operators.

- ANY → true if condition is true for at least one value.
- ALL → true if condition is true for all values.

Example (ANY) : salary greater than any in IT

```
SELECT name, salary
FROM employees
WHERE salary > ANY (SELECT salary
                    FROM employees
                    WHERE dept_id = 10);
```

Example (ALL) : salary greater than all in HR

```
SELECT name, salary
FROM employees
WHERE salary > ALL (SELECT salary
                   FROM employees
                   WHERE dept_id = 20);
```



ANY is like > MIN(value)
ALL is like > MAX(value)

★ 5. SUBQUERY WITH COUNT / IN / NOT IN

Useful for filtering using lists of values.

Using IN

Example :

```
SELECT name
FROM employees
WHERE dept_id IN
      (SELECT dept_id
      FROM departments
      WHERE location = 'Pune');
```

Using NOT IN

Example :

```
SELECT name
FROM employees
WHERE dept_id NOT IN
      (SELECT dept_id
      FROM departments
      WHERE location = 'Pune');
```



Be careful with NOT IN if subquery returns NULL values. It may return no rows!

★ 4. NESTED SUBQUERIES

A subquery inside another subquery.

Example : employees who earn more than average salary of their own department

```
SELECT name, salary, dept_id
FROM employees e
WHERE salary > (SELECT AVG(salary)
               FROM employees
               WHERE dept_id = e.dept_id);
```



Inner subquery gets avg salary for each department.
Outer query compares each employee with that avg.

QUICK TIPS

- Use subqueries to break large problems into small steps.
- Prefer JOIN for better performance when possible.
- Use EXISTS instead of COUNT(*) > 0 for existence checks.
- Always test subqueries separately first.
- Keep subqueries simple and purpose-driven.

WHEN TO USE SUBQUERIES ?

- ✓ When the logic cannot be written easily with JOIN.
- ✓ When you need a value from another query.
- ✓ When you need to filter based on aggregated results.

Great queries start with small steps. Subqueries make complex logic simple! ❤️

11

SUBQUERIES – MORE EXAMPLES

More practical examples using subqueries in real interview scenarios.

★ 1. EMPLOYEES EARNING MORE THAN COMPANY AVERAGE SALARY

Example :

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary)
                FROM employees);
```

Output :

name	salary
Aman	70000
Rohan	80000
Sneha	65000

Compares each employee salary with overall average.



★ 2. DEPARTMENTS WITH MORE THAN 5 EMPLOYEES

Example :

```
SELECT dept_name
FROM departments d
WHERE (SELECT COUNT(*)
       FROM employees e
       WHERE e.dept_id = d.dept_id) > 5;
```

Output :

dept_name
IT
HR
Finance

Counts employees in each dept and filters > 5.



★ 3. EMPLOYEES NOT IN THE IT DEPARTMENT

Example :

```
SELECT name
FROM employees
WHERE dept_id NOT IN
      (SELECT dept_id FROM departments
       WHERE dept_name = 'IT');
```

Output :

name
Priya
Rohan
Sneha
Karan

Excludes employees belonging to IT dept.



★ 4. LATEST SALARY HIKE FOR EACH EMPLOYEE

Assume table: salary_hikes (emp_id, hike_date, new_salary)

Example :

```
SELECT sh1.emp_id, sh1.new_salary, sh1.hike_date
FROM salary_hikes sh1
WHERE sh1.hike_date = (
    SELECT MAX(hike_date)
    FROM salary_hikes sh2
    WHERE sh2.emp_id = sh1.emp_id);
```



Finds the most recent hike date for each employee and shows it.



★ 5. EMPLOYEES WHO EARN MORE THAN THE HIGHEST SALARY IN FINANCE DEPT.

Example :

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT MAX(salary)
                FROM employees e
                JOIN departments d ON e.dept_id = d.dept_id
                WHERE d.dept_name = 'Finance');
```

Output :

name	salary
Aman	70000
Rohan	80000

Compares with highest salary in Finance dept.



★ 6. EMPLOYEES WHO JOINED AFTER THE AVERAGE JOINING DATE

Example :

```
SELECT name, join_date
FROM employees
WHERE join_date > (SELECT AVG(join_date)
                  FROM employees);
```

Output :

name	join_date
Rohan	2022-06-15
Sneha	2022-08-21
Karan	2023-01-10

Filters employees who joined later than the average join date.



IMPORTANT NOTES

- Subqueries can be used in SELECT, FROM, WHERE, HAVING clauses.
- Always check if subquery returns single value or multiple rows.
- Use subqueries to make complex problems simple and readable.



COMMON MISTAKES

- Using > with subquery that returns multiple rows.
- Forgetting to correlate in correlated subqueries.
- Writing unnecessary subqueries (use JOIN when possible).

PRO TIP

Try to think of a problem, write it using JOIN and then try solving it using subqueries. This builds real understanding!



12

SUBQUERIES – FINAL EXAMPLES

Even more real-world examples to strengthen your interview skills!

★ 1. 2nd HIGHEST SALARY FROM EMPLOYEES

Find the second highest salary from employees.

Example :

```
SELECT MAX(salary)
FROM employees
WHERE salary < ( SELECT MAX(salary)
                FROM employees );
```

The inner query gets the highest salary, outer gets the next one. 😊

salary
90000
80000
70000
60000

→ 2nd Highest

★ 2. EMPLOYEES WHO EARN MORE THAN AVG SALARY OF THEIR DEPARTMENT

Find employees who earn more than the average salary of their own department.

Example :

```
SELECT e.name, e.salary, e.dept_id
FROM employees e
WHERE e.salary > ( SELECT AVG(salary)
                  FROM employees
                  WHERE dept_id = e.dept_id );
```



Compares each employee salary with the average salary of their own department. 😊

★ 3. EMPLOYEES WHO EARN MORE THAN THE COMPANY AVERAGE

Example :

```
SELECT name, salary
FROM employees
WHERE salary > ( SELECT AVG(salary)
                FROM employees );
```

Inner query gets company average, outer filters employees. 😊

avg_salary
62500

★ 4. DEPARTMENTS THAT HAVE NO EMPLOYEES

Example :

```
SELECT dept_name
FROM departments d
WHERE NOT EXISTS ( SELECT 1
                  FROM employees e
                  WHERE e.dept_id = d.dept_id );
```

NOT EXISTS checks that no employee is linked to the department. 😊

dept_name
Legal
Admin

★ 5. EMPLOYEES WHO JOINED AFTER THE AVERAGE JOIN DATE

Example :

```
SELECT name, join_date
FROM employees
WHERE join_date > ( SELECT AVG(join_date)
                  FROM employees );
```

Output :

name	join_date
Sneha	2022-08-21
Karan	2023-01-10

Filters employees who joined after the average join date. 😊

★ 6. DEPARTMENTS WITH HIGHEST AVERAGE SALARY

Example :

```
SELECT dept_name
FROM departments
WHERE dept_id = ( SELECT dept_id
                  FROM employees
                  GROUP BY dept_id
                  ORDER BY AVG(salary) DESC
                  LIMIT 1 );
```

Find dept with the highest average salary. 😊

SUBQUERY vs JOIN

SUBQUERY	JOIN
More readable for complex conditions	Better performance in many cases
Good for filtering and aggregation	Good for combining large datasets
Evaluated step by step	Executed in a single pass
Can be nested	Not nested, but can chain joins

TIPS TO REMEMBER

- Start solving problems using subqueries.
- Understand what the inner query returns first.
- Keep subqueries small, readable and meaningful.
- Practice daily with different scenarios! 😊

SUMMARY

- ✓ Subqueries make SQL powerful.
- ✓ They help solve problems that JOINS alone cannot.
- ✓ Practice more to build confidence.
- ✓ Break it down. Think in steps.
- ✓ You've got this! ❤️



Small Queries Today, Big Solutions Tomorrow!



13

SUBQUERIES – FINAL EXAMPLES (PART 2)

More interview level examples to make you even stronger!

★ 1. EMPLOYEES WITH SALARY HIGHER THAN EMPLOYEE 'ROHAN'

Example :

```
SELECT name, salary
FROM employees
WHERE salary > ( SELECT salary
                  FROM employees
                  WHERE name = 'Rohan');
```

Compares each employee's salary with Rohan's salary. 😊

name	salary
Aman	70000
Priya	85000
Sneha	90000

★ 2. DEPARTMENTS WITH MORE EMPLOYEES THAN THE 'HR' DEPARTMENT

Example :

```
SELECT d.dept_name
FROM departments d
WHERE ( SELECT COUNT(*)
        FROM employees e
        WHERE e.dept_id = d.dept_id ) >
      ( SELECT COUNT(*) FROM employees
        WHERE dept_name = 'HR' );
```

Finds departments having more employees than HR department. 😊

dept_name
IT
Finance

★ 3. EMPLOYEES WHO EARN THE HIGHEST SALARY IN THEIR DEPARTMENT

Example :

```
SELECT e.name, e.salary, e.dept_id
FROM employees e
WHERE e.salary = ( SELECT MAX(salary)
                  FROM employees
                  WHERE dept_id = e.dept_id);
```

Gets employees who are top earners in their own department. 😊

name	dept_id	salary
Aman	1	70000
Sneha	2	90000
Rohan	3	80000

★ 4. DEPARTMENTS WITHOUT ANY EMPLOYEES

Example :

```
SELECT d.dept_name
FROM departments d
WHERE NOT EXISTS ( SELECT 1
                   FROM employees e
                   WHERE e.dept_id = d.dept_id);
```

NOT EXISTS is best for checking absence of records. 😊

dept_name
Legal

★ 5. EMPLOYEES WHO JOINED AFTER THE LATEST JOIN DATE IN FINANCE DEPARTMENT

Example :

```
SELECT name, join_date
FROM employees
WHERE join_date > ( SELECT MAX(join_date)
                   FROM employees
                   WHERE dept_id = ( SELECT dept_id
                                     FROM departments
                                     WHERE dept_name = 'Finance' );
```

Nested subquery finds the latest join date in Finance dept, then compares all employees. 😊

name	join_date
Priya	2022-09-01
Karan	2023-02-15

★ 6. EMPLOYEES WORKING IN DEPARTMENTS LOCATED IN 'PUNE'

Example :

```
SELECT e.name, d.dept_name, d.location
FROM employees e
WHERE e.dept_id IN ( SELECT dept_id
                    FROM departments
                    WHERE location = 'Pune');
```

Uses IN with subquery to filter based on department location. 😊

name	dept_name	location
Aman	IT	Pune
Priya	HR	Pune

SUBQUERY PATTERNS

- Single value → =, >, <, >=, <=, <>
- Multiple values → IN, NOT IN
- Existence check → EXISTS, NOT EXISTS
- Aggregation → MAX(), MIN(), COUNT(), AVG()
- Correlated → compares row by row



INTERVIEW TIP

Write the inner query first, check the result, then plug it into the outer query. This avoids many mistakes! 😊

KEY TAKEAWAYS

- Subqueries help break complex problems.
- They can be used in SELECT, FROM, WHERE, HAVING clauses.
- Always check if subquery returns single value or multiple values.
- Practice daily to build confidence!



Practice makes perfect. Keep practicing subqueries and rock your SQL interviews!



14

SUBQUERIES – BEST PRACTICES

Follow these tips to write clean, efficient and correct subqueries.

★ 1. KEEP SUBQUERIES SMALL & FOCUSED

Each subquery should do one job.

Example :

```
SELECT name, salary
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Subquery does one thing → finds average salary. 😊

★ 2. CHOOSE THE RIGHT OPERATOR

Different operators work best in different situations.

Operator	Use When
IN	Match any value in list
NOT IN	Exclude values in list
EXISTS	Check if any row exists
NOT EXISTS	Check if no row exists
ANY	Compare with any value
ALL	Compare with all values



Pick the operator that matches your logic! 😊

★ 3. USE EXISTS / NOT EXISTS FOR PRESENCE CHECKS

EXISTS is usually faster and more reliable than IN for large datasets.

Example :

```
SELECT name
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM departments d
    WHERE d.id = e.dept_id
);
```

Returns employees who belong to any department. 😊

★ 4. AVOID CORRELATED SUBQUERIES WHEN POSSIBLE

Correlated subqueries run once per row.

Try rewriting using JOIN or derived tables.

Example (Correlated) :

```
SELECT name, salary
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);
```

Runs inner query for each row of outer query. 😞

Better (Using JOIN) :

```
SELECT e.name, e.salary
FROM employees e
JOIN (SELECT dept_id, AVG(salary) AS avg_sal
      FROM employees
      GROUP BY dept_id) avg
ON e.dept_id = avg.dept_id
WHERE e.salary > avg.avg_sal;
```

More efficient! 😊

★ 5. BE CAREFUL WITH NULLS

- IN / NOT IN with NULL can return unexpected results.
- Use EXISTS / NOT EXISTS when NULLs are possible.

Example :

```
-- NOT IN may fail if subquery returns NULL
SELECT name
FROM employees
WHERE dept_id NOT IN (SELECT id FROM departments);
```

Better :

```
SELECT name
FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM departments d
    WHERE d.id = e.dept_id
);
```



NOT IN + NULL = No Results!

WHEN TO USE WHAT ?

- Use IN / NOT IN → When comparing with a list of values.
- Use EXISTS / NOT EXISTS → When checking presence / absence.
- Use ANY / ALL → When comparing with ranges or conditions.
- Use derived tables / JOIN → For better performance and when subquery is complex.

COMMON MISTAKES

- ✗ Using NOT IN with NULL values.
- ✗ Writing complex correlated subqueries unnecessarily.
- ✗ Forgetting to test subqueries separately.
- ✗ Not using meaningful aliases.
- ✗ Returning multiple columns when only one value is expected. 😞

QUICK RECAP

- ✓ Keep subqueries simple and focused.
- ✓ Prefer EXISTS over IN for presence checks.
- ✓ Handle NULLs carefully.
- ✓ Use the right operator for the job.
- ✓ Avoid correlated subqueries when possible.
- ✓ Test subqueries separately.

Great SQL is not about how complex your query is, but how simple, correct and efficient your logic is! ❤️



15

SUBQUERIES - ADVANCED TIPS & SUMMARY

Master these advanced tips and remember the big picture of subqueries.

★ 1. SUBQUERY IN UPDATE

Use a subquery to update values.

Example : increase salary by 10% for employees in IT dept.

```
UPDATE employees e
SET salary = salary * 1.10
WHERE dept_id = (SELECT dept_id
FROM departments
WHERE dept_name = 'IT');
```

Subquery finds IT dept_id, outer query updates rows. 😊

★ 2. SUBQUERY IN DELETE

Use a subquery to delete specific rows.

Example : delete employees not in IT or HR dept.

```
DELETE FROM employees
WHERE dept_id NOT IN (
SELECT dept_id
FROM departments
WHERE dept_name IN ('IT', 'HR'));
```

Deletes employees whose dept_id is not in IT or HR. 😊

★ 3. SCALAR SUBQUERY vs. MULTI-ROW SUBQUERY

SCALAR SUBQUERY	MULTI-ROW SUBQUERY
Returns single value	Returns multiple rows
Use with =, <, >, <=, >=, <>	Use with IN, NOT IN, EXISTS, NOT EXISTS
Example returns one salary	Example returns list of dept_ids
Faster and simpler	More flexible and powerful



Choose the right type based on the number of values you expect! 😊

★ 4. SUBQUERIES vs. JOINS

Aspect	SUBQUERY	JOIN
Readability	Good for simple filters	Better for large datasets
Performance	Can be slower (depends on DB)	Usually faster
Flexibility	Very flexible	Less flexible
Use Case	Filtering, existence checks, aggregation	Combining related data from tables

Both are important. Use whatever makes your query clean and efficient! 😊

★ 5. NESTED SUBQUERIES (MULTI-LEVEL)

Subquery inside a subquery.

Example : employees earning more than the average salary of the departments where average salary > 60000.

```
SELECT name, salary
FROM employees
WHERE salary > (
SELECT AVG(avg_sal)
FROM (
SELECT AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 60000
) AS dept_avgs
);
```

First inner subquery gets avg salary of each dept having avg > 60000. Outer compares employee salary with that avg. 😊

★ 6. SUBQUERY WITH WINDOW FUNCTIONS

Subqueries can work with window functions too!

Example : Employees whose salary is more than the department's average using window function in subquery.

```
SELECT name, salary, dept_id
FROM employees e
WHERE salary > (
SELECT AVG(salary) OVER (PARTITION BY dept_id)
FROM employees
WHERE dept_id = e.dept_id
LIMIT 1
);
```

Window function calculates avg per department. Outer query compares. 😊

★ 7. PERFORMANCE TIPS

- Avoid subqueries in SELECT for large datasets.
- Prefer JOINS for heavy data retrieval.
- Use EXISTS instead of IN for better performance.
- Always index columns used in subqueries.
- Test and analyze query execution plan.



Good indexes + smart subqueries = Fast queries! 😊

SUBQUERY CHECKLIST

- ✓ Does the subquery return the expected result?
- ✓ Is the operator correct (=, IN, EXISTS)?
- ✓ Is performance acceptable?
- ✓ Can JOIN be a better option?
- ✓ Are NULLs handled? 😊



WHEN TO AVOID SUBQUERIES

- ✗ When the same data is used multiple times (use JOIN).
- ✗ When the subquery is returning too many rows.
- ✗ When it makes the query hard to read or maintain.
- ✗ When performance is critical and JOIN is clearly faster. 😊

FINAL SUMMARY



Subqueries help filter, compare, aggregate and manipulate data.



Use the right operator for the right job.



Practice more complex examples to build confidence and skills.



Clean logic, good performance and clear queries win interviews! ❤️

Understand. Practice. Optimize. That's how you become a SQL Pro! ❤️